ETロボコン Unity シミュレータ
新しい PC への導入と日々の使い方

2026-09-17 版 ／ 詳細は EtroboUnitySim/README.md を参照

1. 必要なもの

ソフト	バージョン	備考
Windows	10 / 11（64bit）	
Unity Hub Unity	2022.3 LTS （確認済み 2022.3.41f1）	外部アセットは使わない。コースも走行体も寸法から生成する
Visual Studio	2022 以降	ワークロード「**C++ によるデスクトップ開発**」が必須。 CMake は VS 同梱のものを自動で使うので PATH 設定は不要
Python	3.8 以上（任意）	コース変換ツールとヘッドレス版シミュレータ用。走らせるだけなら不要
Git	任意	チームの走行プログラムを clone する場合

- コース図 PDF からコースを作り直すときだけ pip install pymupdf が要ります。
- 通信は **localhost の UDP 54001 のみ**。外には出ないのでファイアウォールの許可は通常不要です。
- ウイルス対策ソフトが自作 exe を検疫することがあります（署名なし・毎回ビルドし直す・ UDP ソケットを開く、が揃うため）。必要なら controller¥bin と controller¥bin_eyes を除外に登録してください（設定変更はご自身の判断で）。

2. 置き場所

```
C:¥Users¥<ユーザ>¥Documents¥MyProjects¥
 ├── EtroboUnitySim¥          シミュレータ本体
 │    ├── controller¥          制御プロセス（C/C++）
 │    ├── course¥              コース JSON・走行体パラメータ
 │    ├── protocol¥            C と C# が共有する通信仕様
 │    ├── tools¥               Python ツール
 │    └── unity¥EtroboSim¥     Unity プロジェクト
 └── eyes-et2026¥             チームの走行プログラム（git clone）
```

置き場所は自由ですが、パスに空白や日本語を含めないほうが無難です。

コピーしなくてよいもの（新しい PC で作り直されます）

- unity¥EtroboSim¥ の Library Temp Obj Logs UserSettings、*.csproj、*.sln
- controller¥ の build build_eyes bin bin_eyes

これらを除くと 25MB 程度です（Library を含めると 66MB）。 Library は Unity が初回起動時に作り直すので、コピーしても速くなりません。

3. 新しい PC への移し方

1. **Unity Hub をインストール**し、Unity **2022.3 LTS** を追加する。
2. **Visual Studio 2022 以降をインストール**する。ワークロードは 「C++ によるデスクトップ開発」。
3. **EtroboUnitySim フォルダをコピー**する（上の「コピーしなくてよいもの」は除く）。
4. **チームのコードを取得**する。

```
git clone https://github.com/m-sugimoto-eyes/eyes-et2026.git
```

5. **制御プロセスをビルド**する。

```
cd <EtroboUnitySim>¥controller
.¥build.ps1
```

controller¥bin¥etrobo_app.exe ができます（同梱のサンプル走行プログラム）。

6. **Unity プロジェクトを開く。**Unity Hub → Add → Add project from disk で <EtroboUnitySim>¥unity¥EtroboSim を追加して開く。 初回は Library の再生成で数分かかります。
7. **移行できたか確かめる。**Unity のメニュー **「ETロボコン → 自己テストを実行」**を実行し、最後に 「すべて通りました」と出れば成功です。

4. 自分たちのプログラムを更新して反映する

チームのソースには手を入れずにそのまま動きます。

1. **最新を取る。**

```
cd <eyes-et2026>
git pull
```

2. **初回だけ CMake の構成をする。**（PowerShell）

```
cd <EtroboUnitySim>¥controller
$eyes = "<eyes-et2026 のパス>"
cmake -S . -B build_eyes -DETROBO_APP=eyes2026 -DEYES2026_DIR=$eyes -DETROBO_BIN_DIR=bin_eyes
```

cmake に PATH が通っていないときは VS 同梱のものを使います（cmake の代わりに & $cmake と書く）。
$cmake = "C:¥Program Files¥Microsoft Visual Studio¥<版>¥<エディション>¥Common7¥IDE¥CommonExtensions¥Microsoft¥CMake¥CMake¥bin¥cmake.exe"

3. **2 回目以降はビルドだけ。**ソースを直したらここだけ回します。

```
cmake --build build_eyes --config Release
```

controller¥bin_eyes¥etrobo_app.exe が更新されます。

4. **Unity を再生状態にしてから、走行プログラムの置き場で**実行する。

```
cd <eyes-et2026>
<EtroboUnitySim>¥controller¥bin_eyes¥etrobo_app.exe
```

Scenario.csv を相対パスで読むので、置き場で実行するのが肝心です。

5. **前の制御プロセスが残っていないか確かめる。**残っていると 2 つが同時に指令を送り、 テレメトリが 0 のまま固まるなど紛らわしい状態になります。

5. Unity で Play する

1. Unity で Assets/Scenes/SimMain.unity を開く。
2. **再生ボタン（▶）を押す。**コース・走行体・カメラ・HUD は再生時に生成されます （シーンに置いてあるのは SimBootstrap 1 個だけ）。
3. 別のターミナルで**制御プロセスを起動する。**HUD の「制御プロセス」が **接続中**に変わります。
4. パネルの **中央ボタン** か **フォース**、またはキーボードの **Space** でスタートします。

キー	動作
Space	スタートの合図（中央ボタンとフォースセンサの両方に入る）
R	走行体を**開始位置**へ戻す（ボトルも設定した位置へ立て直す）

P	配置モード。矢印キーで走行体を動かして開始位置を決める
Tab	視点モードの切り替え
F / G	走行体に寄る / コース全体を映す

パネル（HUD）

走行中の状態と設定はパネルにまとまっています。扱えるのは、 **コースの L/R 切り替え**、スタートの合図、開始位置、キャリーボトルの色、 ゲートと力士ボトルの配置、難所とフィニッシュの成立判定です。 パネルは走行体が出るのと反対側に出ます。

- 行が多いので、**見出し（- センサ など）をクリックすると節をたためます。** たたんだ状態は次に起動したときも残ります。収まらないぶんはパネルの中をスクロールできます。
- カーソルがパネルの上にある間はカメラが動かないので、 ドラッグやホイールで視点が変わってしまうことはありません。

任意の位置から始める

難所の手前だけを何度も試したいときは開始位置を動かします。 R で戻る場所が変わるだけで、走らせ方は今までと同じです。

- **矢印キーで合わせる** – P か パネルの 配置(P) で配置モードへ。 ↑↓ で前後、←→ で左右、 カンマ , で左回り、ピリオド . で右回り （Shift を押しながらで 10 倍）。 座標を計算せずに「スタートラインのぎりぎり手前」を出せます。
- **走らせた場所を使う** – 現在地に で、いま走行体がいる場所が開始位置になります。
- **コースに焼く** – 保存 でコース JSON の start_pose を書き換えます （StreamingAssets と course/ の両方）。
- **起動時に指定** – -etsimStart X,Y[,方位]。方位は 0 が +y、90 が -x です。

計測ゲートは動きません。スタートラインより先に置くと計測が始まらないので、 タイムを見たいときはコース既定の位置から走らせてください。
コースを固定で変えたいときは SimBootstrap の **Course File** を 書き換えます（既定は etrobo_2026_L.json）。

6. よくあるつまずき

症状	見るところ
ビルドは通るのに LNK1104／実行した exe が消える	ウイルス対策ソフトの検疫。controller¥bin を除外に登録する
Unity 側が「待機中」のまま	制御プロセスが起動していない／ポート違い／前のプロセスが残っている
走行体がその場で回る	モータの極性かポート割り当ての取り違え。 既定は A=右 B=左 C=アーム D=フォース E=カラー F=超音波
制御プロセスの日本語が化ける	パイプやファイルに流したとき。PowerShell なら [Console]::OutputEncoding = [Text.Encoding]::UTF8、ファイルは Get-Content -Encoding UTF8
難所（ボトル・ゲート）が出ない	難所の入ったコースか確認する（_nansyo が付いたもの）。起動ログの props=5/5 が**組み立てた数／JSON の数**
新しく足した .cs が反映されない	Unity を開き直す。初回のアセット取り込みに間に合わないことがある

詳しくは README.md の「9. うまくいかないとき」を参照してください。